

Project WILSON

Διαδραστικός Προσομοιωτής Εξάπλωσης Δασικών Πυρκαγιών με Βαθμονόμηση σε
Πραγματικά Ελληνικά Συμβάντα

Αναλυτική τεχνική αναφορά — με έμφαση στην υπολογιστική όραση και την τρισδιάστατη
γεωμετρία

Ιούλιος 2026

Contents

1	Εισαγωγή και στόχος του συστήματος	3
2	Πυρήνας φυσικής προσομοίωσης (core/)	3
2.1	Μοντέλο φωτιάς core/fire_model.py	3
2.2	Τοπίο και καύσιμη ύλη core/landscape.py, core/fuels.py	3
2.3	Πεδίο ανέμου air/air.py	4
2.4	Μικροκλιματική μνήμη core/microclimate.py	4
3	Δεδομένα και pipelines απόκτησης (pipeline/)	4
3.1	Αυτόματη λήψη δεδομένων pipeline/auto_fetcher.py	4
3.2	Ιστορικές περιμέτροι pipeline/fired_loader.py	5
3.3	Καιρικά δεδομένα (ERA5)	5
4	Βαθμονόμηση και αντίστροφη μοντελοποίηση	5
4.1	Sandbox βελτιστοποιητής PSO pipeline/particle_optimizer.py	5
4.2	Hindcast βελτιστοποιητής pipeline/hindcast_optimizer.py	6
4.3	Αποτελέσματα επικύρωσης (σύνοψη)	6
5	Υπολογιστική όραση: ανίχνευση φωτιάς από drone (drone/)	6
5.1	Στάδιο 1: Ανιχνευτής YOLOv8 (drone/src/vision.py)	7
5.2	Στάδιο 2α: Χρωματομετρική κατάτμηση RGB/YCbCr («Bravonoid»)	8
5.3	Στάδιο 2β: Θερμική (IR) κατάτμηση	9
5.4	Στάδιο 3: Μορφολογία, πολύγωνο και οπτική τηλεμετρία	10
5.5	Επιδόσεις όρασης	10
6	Τρισδιάστατη γεωμετρία I: γεωαναφορά pixel → GPS (drone/src/geometry.py)	10
6.1	Συστήματα συντεταγμένων και συμβάσεις	10
6.2	Βήμα 1: Αντιστροφή του μοντέλου pinhole	11
6.3	Βήματα 2–3: Αλυσίδα στροφών cam → body → NED	12
6.4	Βήμα 4: Τομή ακτίνας με το έδαφος (flat-earth)	12
6.5	Βήμα 5: Μετρικές μετατοπίσεις → WGS-84	12
6.6	Βήμα 6: Ευρετική εμπιστοσύνη και ανάλυση σφάλματος	13

6.7	Τηλεμετρία DJI: από flight record σε γωνίες κάμερας (drone/src/dji_log.py)	13
6.8	Ενορχήστρωση και καταγραφή (drone/src/main.py)	14
7	Τρισδιάστατη γεωμετρία II: ανακατασκευή και απεικόνιση αναγλύφου	15
7.1	Flask API πλεγμάτων mesh_api.py	15
7.2	Live 3D προβολή προσομοίωσης simulation_3d_view.py, viz/	15
8	Server προσομοίωσης server.py	17
9	Web διεπαφή sandbox.html	17
10	Σύμβουλος απειλών (Threat Advisor) pipeline/threat_advisor.py	17
11	Εκκίνηση και βοηθητικά	18
12	Επικύρωση, περιορισμοί και μελλοντικές επεκτάσεις	18
12.1	Τι έχει μετρηθεί	18
12.2	Γνωστοί περιορισμοί	18
12.3	Πιθανές επεκτάσεις	18
A	Παράρτημα A: Πίνακες μετρήσεων	18
B	Παράρτημα B: Οδηγίες αναπαραγωγής σχημάτων	18

1 Εισαγωγή και στόχος του συστήματος

Το WILSON είναι ένα πλήρες σύστημα προσομοίωσης εξάπλωσης δασικής πυρκαγιάς σε πραγματικό ελληνικό ανάγλυφο. Συνδυάζει: (α) φυσική Rothermel πάνω σε κυψελωτό αυτόματο (cellular automaton), (β) πραγματικά δεδομένα εδάφους, καύσιμης ύλης και καιρού, (γ) διαδραστικό χάρτη στον browser με εργαλεία επέμβασης (αντιπυρικές ζώνες, ρίψεις νερού, γραμμές ανάσχεσης), (δ) δύο ανεξάρτητους βελτιστοποιητές βαθμονόμησης (Particle Swarm Optimization και Differential Evolution), (ε) επικύρωση (hindcast) έναντι δορυφορικών περιμέτρων πραγματικών πυρκαγιών (Εύβοια 2021, Ρόδος 2023, Έβρος 2023), (στ) ένα πειραματικό υποσύστημα *υπολογιστικής όρασης από drone* που εντοπίζει ενεργή φλόγα σε βίντεο και τη γεωαναφέρει σε συντεταγμένες WGS-84 μέσω μοντέλου pinhole κάμερας, και (ζ) τρισδιάστατη απεικόνιση αναγλύφου με διορθωμένο από το έδαφος πεδίο ανέμου.

Αρχιτεκτονικά, το σύστημα αποτελείται από Python backend (WebSocket server, port 8765), web client (MapLibre, `sandbox.html`), βοηθητικό Flask API για τρισδιάστατα πλέγματα (`mesh_api.py`, port 5000) και το αυτόνομο drone pipeline (`drone/src/`). Η παρούσα αναφορά περιγράφει όλα τα υποσυστήματα, με ιδιαίτερο βάθος στα κεφάλαια 5 (υπολογιστική όραση) και 6 (τρειςδιάστατη γεωμετρία), και παραθέτει μετρήσεις επιδόσεων· ο πλήρης πίνακας δεδομένων επικύρωσης βρίσκεται στο συνοδευτικό `report_data.tex`.

2 Πυρήνας φυσικής προσομοίωσης (core/)

2.1 Μοντέλο φωτιάς `core/fire_model.py`

Ο πυρήνας είναι η κλάση `CellularAutomataFire` (~900 γραμμές, πλήρως vectorised NumPy). Κάθε κελί του πλέγματος βρίσκεται σε μία από τρεις καταστάσεις (0 άκαυτο, 1 καιγόμενο, 2 καμένο) και συσσωρεύει θερμότητα στο πεδίο `ignition_fraction`· όταν αυτό ξεπεράσει το ρυθμιζόμενο κατώφλι ανάφλεξης (`_ignition_threshold`), το κελί αναφλέγεται και καίει για `_burn_time_steps` βήματα (κλιμακούμενα με το χρονικό βήμα dt).

Η μέθοδος `_precompute_ros_grid()` υλοποιεί την πλήρη εξίσωση Rothermel (1972) ανά κελί — ένταση αντίδρασης (reaction intensity), λόγος συμπίεσης (packing ratio), απόσβεση υγρασίας η_m , παράγοντας ανέμου ϕ_w (με όριο 1500 ft/min) και παράγοντας κλίσης ϕ_s — και παράγει πίνακα πιθανοτήτων εξάπλωσης `r_spread` διαστάσεων $8 \times R \times C$ (8 γείτονες). Σημαντική λεπτομέρεια υλοποίησης: κάθε ανακατασκευή του πίνακα (π.χ. σε ωριαία ενημέρωση καιρού) μηδενίζει εξωτερικές τροποποιήσεις, γεγονός που επηρέασε τη σχεδίαση της βαθμονόμησης (§4).

Επιπλέον μηχανισμοί:

- *Monte-Carlo spotting* (τύπου Albini): πιθανότητα εκτόξευσης καύτρας ανάλογη της ταχύτητας ανέμου, log-normal κατανομή απόστασης πτήσης, απόσβεση από την τοπική ισχύ γραμμών ανάσχεσης στο σημείο πτώσης.
- *Νερό/κατάσβεση*: πιθανοτική κατάσβεση με σιγμοειδή συνάρτηση (ποσότητα νερού έναντι φορτίου καύσιμης ύλης, υγρασίας, ανέμου), περιορισμένη στο bounding box των καιγόμενων κελιών για επιδόσεις.
- *Ντετερμινισμός*: σταθερό RNG seed για αναπαραγωγιμότητα.

Επιδόσεις ανά βήμα προσομοίωσης: 0,9 ms σε πλέγμα 200^2 , 3,5 ms σε 400^2 , 16 ms σε 800^2 .

2.2 Τοπίο και καύσιμη ύλη `core/landscape.py`, `core/fuels.py`

Ορίζονται 23 ελληνικές κλάσεις καύσιμης ύλης (GREEK_FUELS): πεύκα (χαλέπιος, τραχεία κ.λπ.), μακία, φρύγανα, ξηρά χόρτα, ελαιώνες, δρυοδάση κ.ά., με πλήρεις παραμέτρους Rothermel (fuel load, surface-area-to-volume ratio, moisture of extinction, bulk density), συν 4 μη καύσιμες κλάσεις (νερό, αστικός

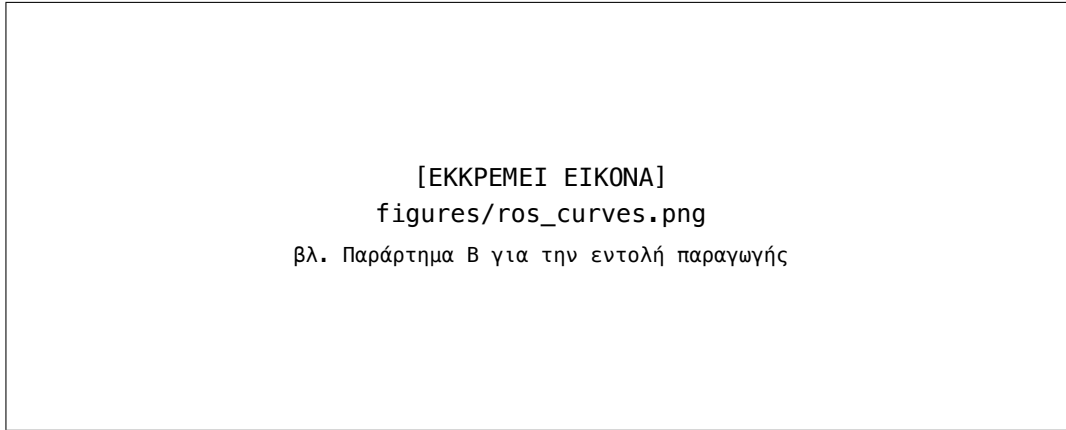


Figure 1: Χαρακτηρισμός φυσικής: ταχύτητα εξάπλωσης (ROS) συναρτήσκει ταχύτητας ανέμου, κλίσης και υγρασίας καύσιμης ύλης, ανά αντιπροσωπευτική κλάση βλάστησης. Το αναλλοίωτο ανάλυσης (100→50 m) παραμένει εντός 3%.

ιστός, δρόμοι, άγρονο). Η `load_real_terrain()` συνθέτει: DEM (GeoTIFF) + CORINE land cover (PNG) χαρτογραφημένο σε κλάσεις καύσιμης ύλης + OSM overlay (δρόμοι → Urban_Road, οικιστικά πολύγωνα → Urban_Fabric) μέσω rasterio. Το πλέγμα είναι south-up (γραμμή 0 = νότος). Η υγρασία της καύσιμης ύλης προκύπτει από το Equilibrium Moisture Content (θερμοκρασία / σχετική υγρασία).

2.3 Πεδίο ανέμου `air/air.py`

Ο άνεμος midflame προκύπτει από τρία διαδοχικά στρώματα διόρθωσης πάνω στον βασικό μετεωρολογικό άνεμο:

1. *Wind Adjustment Factor (WAF)* ανά κλάση βλάστησης: πυκνό δάσος $\approx 0,10\times$, χόρτα $\approx 0,40\times$ (μετρημένο στο validation).
2. *Ορογραφική ενίσχυση*: επιτάχυνση σε ανηφορική ροή με πρόσθετο μπόνους νότιου προσανατολισμού.
3. *Mass-consistent διόρθωση* τύπου WindNinja: διαγνωστική επίλυση Poisson

$$\nabla^2 \phi = \text{div}(U, V), \quad (U', V') = (U, V) - \nabla \phi,$$

με μετασχηματισμό DCT-II (συννοριακές συνθήκες Neumann, πολυπλοκότητα $O(NM \log NM)$), ώστε το τελικό πεδίο να είναι divergence-free: επιτάχυνση σε κορυφογραμμές, συμπίεση ροής σε κοιλάδες.

2.4 Μικροκλιματική μνήμη `core/microclimate.py`

Ο `MicroclimateLearner` διατηρεί rolling average χάρτη συχνότητας καύσης `ignition_freq` από προηγούμενα τρεξίματα, με σειριοποίηση JSON (~1,4 MB, 25 ms, lossless round-trip), `bilinear resize_to()` μεταξύ αναλύσεων και `apply_to_model()` που ενισχύει το `p_spread` σε ιστορικά ενεργούς διαδρόμους. Ενημερώνεται και από τα top-K αποτελέσματα του PSO. Μετρήθηκε ότι επιταχύνει την αρχική σύγκλιση του PSO, αλλά μπορεί να μεροληπτει την τελική βαθμονόμηση.

3 Δεδομένα και pipelines απόκτησης (`pipeline/`)

3.1 Αυτόματη λήψη δεδομένων `pipeline/auto_fetcher.py`

Ενιαίο σημείο λήψης με cache ανά bounding box στον δίσκο:

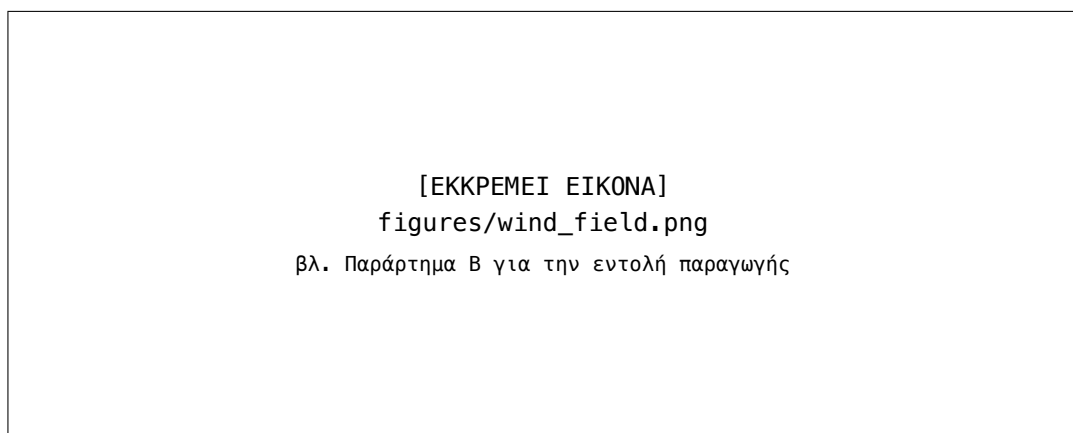


Figure 2: Διορθωμένο πεδίο ανέμου πάνω σε πραγματικό ανάγλυφο (streamlines): αριστερά ομοιόμορφος μετεωρολογικός άνεμος, δεξιά μετά τα τρία στρώματα διόρθωσης (WAF, ορογραφία, mass-consistent Poisson). Διακρίνεται η επιτάχυνση στις κορυφογραμμές.

- DEM: Copernicus COP30 μέσω terrain-tiles API (zoom 14), cache terrain_{lat}_{lon}_{buf}.tif.
- CORINE land cover μέσω WMS σε PNG 200–800 px.
- OSM (Overpass): δρόμοι (motorway...service) και οικιστικά πολύγωνα με πλήρη γεωμετρία (out geom)· τρία mirrors με fallbacks.
- fetch_osm_places(): ονομασμένοι οικισμοί (place=city/town/village/hamlet/suburb) για τον Threat Advisor.

3.2 Ιστορικές περιμέτροι pipeline/fired_loader.py

Φόρτωση FIRED daily perimeters (GeoPackage 22 MB, 24.867 features, Ελλάδα 2000–2024). Γνωστά γεγονότα: Εύβοια 2021 (id 2871, 12 ημέρες, 52,5 kha), Ρόδος 2023 (id 130111, 10 ημέρες, 19,9 kha), Έβρος 2023 (id 105620, 18 ημέρες, 89,4 kha). Η get_day1_ignition_seeds() σπάει MultiPolygon σε ξεχωριστά σημεία έναυσης (π.χ. Εύβοια: 2 εστίες)· η περίμετρος rasterize-άρεται σε μάσκα πλέγματος για scoring.

3.3 Καιρικά δεδομένα (ERA5)

Οι fetch_weather / fetch_weather_hourly (στο hindcast_optimizer.py) αντλούν από το Open-Meteo ERA5 archive: άνεμος 10 m, διεύθυνση, θερμοκρασία, σχετική υγρασία ανά ώρα, με μετατροπή της σύμβασης διεύθυνσης «από» → «προς». Εύρημα validation: το ERA5 0,25° υποεκτιμά συστηματικά τον άνεμο των ημερών έκρηξης κατά 0,4–3 m/s — εξ ου και ο πολλαπλασιαστής ανέμου στη βαθμονόμηση.

4 Βαθμονόμηση και αντίστροφη μοντελοποίηση

4.1 Sandbox βελτιστοποιητής PSO pipeline/particle_optimizer.py

Particle Swarm 12 παραμέτρων που ταιριάζει την προσομοίωση σε χειροκίνητα σχεδιασμένη περίμετρο «αληθείας» στο UI. Χώρος αναζήτησης: πολλαπλασιαστής και στροφή ανέμου, offset υγρασίας, canopy WAF, slope factor, 5 πολλαπλασιαστές ROS ανά ομάδα καύσιμης ύλης, spotting rate, κατώφλι ανάφλεξης. Τρέχει σε γρήγορο πλέγμα 200² με ThreadPoolExecutor· συνάρτηση καταλληλότητας το IoU στη χρονική στιγμή της μάσκας, συν ensemble heatmap πρόβλεψης +1 h από τα top-50 τρεξίματα. Επιστρέφει και το «παγωμένο» state του καλύτερου σωματιδίου, ώστε το «συνέχισε από το best-IoU fire» να

επαναφέρει ακριβώς εκείνη τη φωτιά (nearest-neighbour αναγωγή 200 → 800) αντί να ξανατρέχει στοχαστικά.

Παγίδα υλοποίησης: το `_precompute_ros_grid()` σβήνει κάθε εξωτερική τροποποίηση· ο canopy παράγοντας επιβιώνει πλέον ως attribute (`_canopy_wind_mult`) και οι λοιποί πολλαπλασιαστές επανεφαρμόζονται μετά από κάθε rebuild (ωριαίος καιρός, ρίψεις νερού).

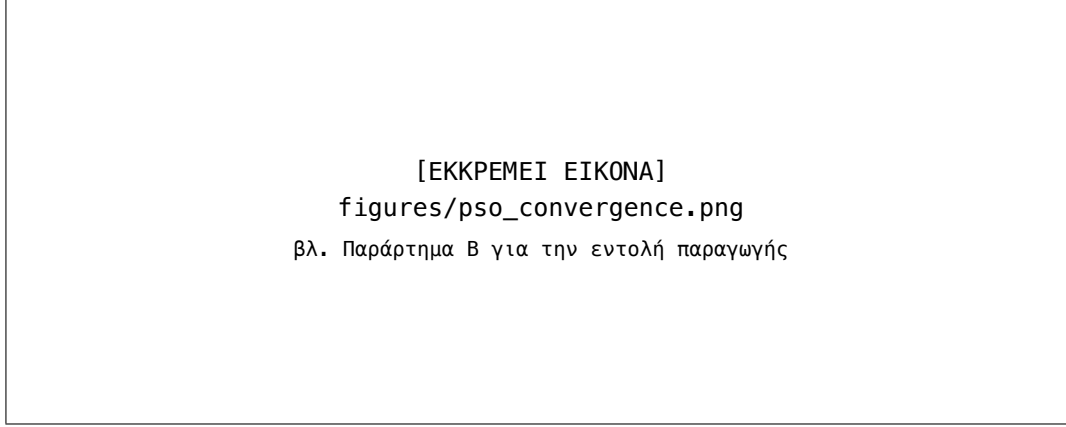


Figure 3: Σύγκλιση PSO: βέλτιστο IoU ανά γενιά για 5 ανεξάρτητα τρεξίματα, με και χωρίς προθέρμανση από τη μικροκλιματική μνήμη.

4.2 Hindcast βελτιστοποιητής pipeline/hindcast_optimizer.py

Πλήρες end-to-end pipeline επικύρωσης σε πραγματικές πυρκαγιές: αλήθεια FIRED/FIRMS, καιρός ERA5, πραγματικό έδαφος, βελτιστοποίηση Differential Evolution σε 5 κρυφές μεταβλητές: offset υγρασίας $[-0,15, 0]$, πολλαπλασιαστής ανέμου $[0,5, 6]$, στροφή ανέμου $\pm 30^\circ$, κατώφλι ανάφλεξης $[0,15, 1]$, πολλαπλασιαστής διάρκειας καύσης $[1, 5]$. Σύνθετο σφάλμα:

$$E = 0,3 (1 - \text{IoU}) + 0,7 H_{\text{norm}},$$

όπου H_{norm} το συμμετρικό Hausdorff κανονικοποιημένο στη διαγώνιο του πλέγματος — δίνει ομαλή κλίση ακόμα και όταν οι μάσκες δεν τέμνονται. Προεπιλογή σε FIRED mode είναι το *perimeter assimilation* (τύπου FARSITE validation): έναυση από τη σωρευτική παρατηρημένη περίμετρο της ημέρας ignition_day και scoring της σωρευτικής έκτασης στο τέλος του παραθύρου· εναλλακτικά point-seed από centroids για «καθαρή» from-ignition δεξιότητα. Η βαθμονόμηση γίνεται σε αδρό πλέγμα 200^2 ($dt = 1 \text{ min}$) και το τελικό IoU υπολογίζεται σε 800^2 με CFL-σωστή κλιμάκωση $dt \propto \Delta x$.

4.3 Αποτελέσματα επικύρωσης (σύννοψη)

Από 13 hindcast τρεξίματα: assimilated ημέρας-4 μέσο IoU $\approx 48\%$ (Εύβοια 53,2%, Ρόδος 47,6%, Έβρος 43,0%)· ημέρας-6/8 έως 66,7–90,2%. Point-seed ημέρας-1 μόνο $\sim 2\%$ όταν η ημέρα 1 είναι σιγοκαίουσα (Εύβοια 43 ha, Έβρος 22 ha), αλλά 42,6% στη Ρόδο (ημέρα 1 ήδη 968 ha). Συστηματική υπερπρόβλεψη 1,1–2,2× (απουσία μοντέλου κατάσβεσης, ομαλοποιημένος ERA5 άνεμος). Ο πλήρης πίνακας στο report_data.tex.

5 Υπολογιστική όραση: ανίχνευση φωτιάς από drone (drone/)

Το υποσύστημα drone υλοποιεί έναν υβριδικό ανιχνευτή ενεργής φλόγας σε ροή βίντεο: ένας βαθύς ανιχνευτής αντικειμένων (YOLOv8) εντοπίζει πού υπάρχει πιθανή φωτιά (bounding box), και στη

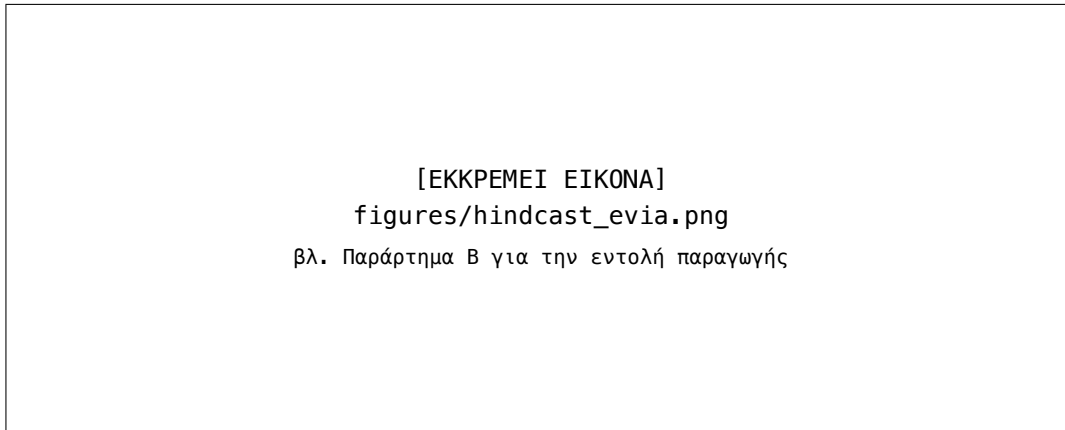


Figure 4: Διαγνωστικό τετράπτυχο hindcast (Εύβοια 2021, ημέρα 4, perimeter assimilation): παρατηρημένη περίμετρος FIRED, προσομοιωμένη έκταση, επικάλυψη (IoU 53,2%), χάρτης σφάλματος. Παράγεται από την `plot_results()`.

συνέχεια κλασική χρωματομετρική κατάτμηση οριοθετεί *ακριβώς* τα pixels της φλόγας μέσα στο box. Ο συνδυασμός δίνει και τη σημασιολογική στιβαρότητα του νευρωνικού δικτύου (απόρριψη ήλιου, φώτων, πορτοκαλί αντικειμένων) και την pixel-ακρίβεια της κατάτμησης, η οποία είναι απαραίτητη για τη γεωαναφορά του κεφαλαίου 6: το κέντρο βάρους του πολυγώνου φλόγας — όχι του χονδρικού box — είναι αυτό που προβάλλεται στο έδαφος.

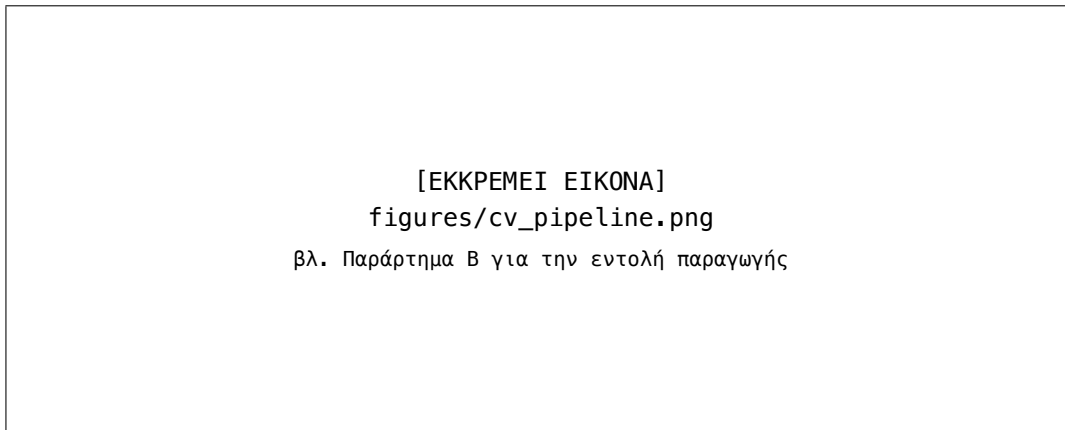


Figure 5: Αρχιτεκτονική του pipeline όρασης: frame → YOLOv8 (πρόταση περιοχών) → χρωματική κατάτμηση Bravonoid (RGB) ή θερμική κατωφλίωση (IR) → μορφολογία → πολύγωνο φλόγας, κέντρο βάρους, ρυθμός οπτικής μεγέθυνσης.

5.1 Στάδιο 1: Ανιχνευτής YOLOv8 (`drone/src/vision.py`)

Χρησιμοποιείται μοντέλο YOLOv8-nano (`fire_yolov8n.pt`, ultralytics) με κατώφλι εμπιστοσύνης `conf=0.3`. Ο ανιχνευτής επιστρέφει boxes με κλάσεις που ελέγχονται *δυναμικά* από το `model.names`: κάθε κλάση που περιέχει τη λέξη «fire» δρομολογείται στη λεπτομερή κατάτμηση, ενώ οι υπόλοιπες (π.χ. «smoke») απλώς σημειώνονται στο πλαίσιο προεπισκόπησης. Στην IR παραλλαγή (`drone/fire_tracker.py`) η συμπερασματολογία τρέχει σε `imgsz=320` για επιδόσεις edge. Τα boxes ψαλιδίζονται στα όρια του frame πριν από την εξαγωγή crop.

Η επιλογή του «πρώτα ανιχνευτής, μετά κατάτμηση» έχει δύο πρακτικά πλεονεκτήματα: (α) η

ακριβή χρωματική ανάλυση τρέχει μόνο στο μικρό crop του box και όχι σε ολόκληρο το frame, και (β) οι χρωματικοί κανόνες, που μόνοι τους θα έδιναν ψευδώς θετικά σε δύση ηλίου ή κόκκινες στέγες, εφαρμόζονται μόνο εκεί όπου το δίκτυο έχει ήδη δώσει σημασιολογική ένδειξη φωτιάς.

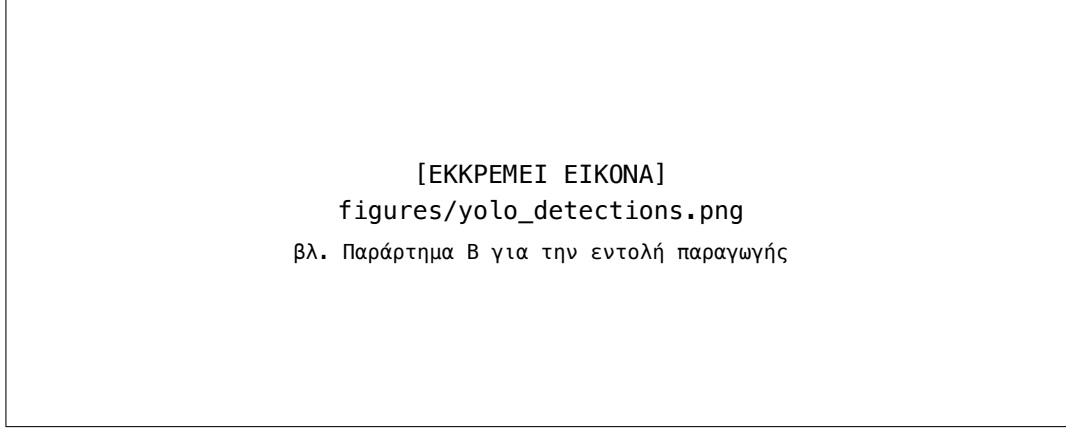


Figure 6: Παραδείγματα ανίχνευσης σε πραγματικό υλικό πτήσης (φάκελος drone/data/vision_check/): επιτυχείς ανιχνεύσεις με box (μπλε), πολύγωνο κατάτμησης (πράσινο) και κέντρο βάρους (κόκκινο), μαζί με δύο αρνητικά frames χωρίς ανίχνευση για έλεγχο ψευδώς θετικών.

5.2 Στάδιο 2α: Χρωματομετρική κατάτμηση RGB/YCbCr («Bravonoid»)

Μέσα στο crop της φωτιάς εφαρμόζεται συνδυασμός εμπειρικών χρωματικών κανόνων σε δύο χρωματικούς χώρους.

Κανόνες RGB. Για κάθε pixel με κανάλια (R, G, B) :

$$R1: R > G > B$$

$$R2: R > 190, G > 90, B < 140$$

$$R3: 0,1 \leq \frac{G}{R+1} \leq 1,0, 0,1 \leq \frac{B}{R+1} \leq 0,85, 0,1 \leq \frac{B}{G+1} \leq 0,85$$

Το R1 κωδικοποιεί τη φασματική διάταξη της φλόγας (κόκκινο > πράσινο > μπλε), το R2 απαιτεί απόλυτη φωτεινότητα στο κόκκινο κανάλι, και το R3 περιορίζει τους λόγους καναλιών ώστε να αποκλείονται κορεσμένα λευκά και γκριζα (ο παρονομαστής +1 αποτρέπει διαίρεση με το μηδέν).

Κανόνες YCbCr. Στον χώρο φωτεινότητας–χρωμικότητας:

$$(Y \geq 170 \vee Y < 145) \wedge 50 \leq C_b \leq 120 \wedge 120 < C_r < 220.$$

Η φλόγα έχει χαρακτηριστικά υψηλό C_r (κόκκινη χρωμικότητα) και χαμηλό C_b : ο διαζευκτικός όρος στο Y πάνει τόσο τον υπέρλαμπρο πυρήνα όσο και τις σκοτεινότερες παρυφές της φλόγας.

Ενίσχυση χαμηλού φωτισμού και συναίνεση. Το crop περνά και από παραλλαγή ενισχυμένη για χαμηλό φωτισμό: ισοστάθμιση ιστογράμματος στο κανάλι V του HSV (equalizeHist) και median blur 5×5 . Οι κανόνες εφαρμόζονται και στις δύο εκδοχές (κανονική/ενισχυμένη), δίνοντας τέσσερις μάσκες: RGB, RGB_{low}, YCbCr, YCbCr_{low}. Συνδυάζονται ως

$$M_U = (RGB \vee YCbCr) \wedge (RGB_{low} \vee YCbCr_{low}), \quad M_{RGB} = RGB \vee RGB_{low},$$

και επιλέγεται η M_{RGB} όταν οι δύο μάσκες συμφωνούν σε $\geq 75\%$ των pixels του crop (ένδειξη «εύκολης» σκηνής), αλλιώς η αυστηρότερη M_U . Ο μηχανισμός αυτός λειτουργεί ως αυτόματη εναλλαγή μεταξύ ευαισθησίας και ακρίβειας ανάλογα με τη δυσκολία της σκηνής.

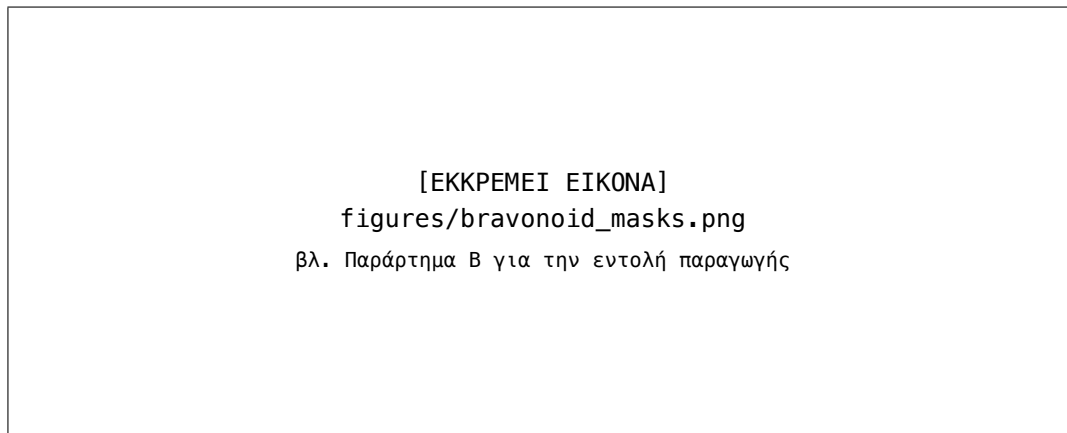


Figure 7: Στάδια της κατάτμησης Bravonoid σε ένα crop φωτιάς: αρχικό crop, ενισχυμένη low-light εκδοχή, μάσκα RGB, μάσκα YCbCr, τελική μάσκα συναίνεσης, και το τελικό πολύγωνο μετά τη μορφολογία.

5.3 Στάδιο 2β: Θερμική (IR) κατάτμηση

Για θερμικές κάμερες σε λειτουργία «White Hot» (drone/fire_tracker.py, CAMERA_MODE='IR') η χρωματική ανάλυση αντικαθίσταται από κατωφλίωση έντασης: μετατροπή σε grayscale (αν η κάμερα στέλνει pseudo-color παλέτα π.χ. Ironbow), Gaussian blur 5×5 για τον έντονο θόρυβο των αισθητήρων μικροβολομέτρων, και δυαδική κατωφλίωση στο 230/255 — η φωτιά είναι απλώς τα θερμότερα pixels. Το κατώφλι είναι ρυθμιζόμενο ανά κάμερα, αφού η αντιστοίχιση θερμοκρασίας–έντασης εξαρτάται από το AGC της κάθε συσκευής.

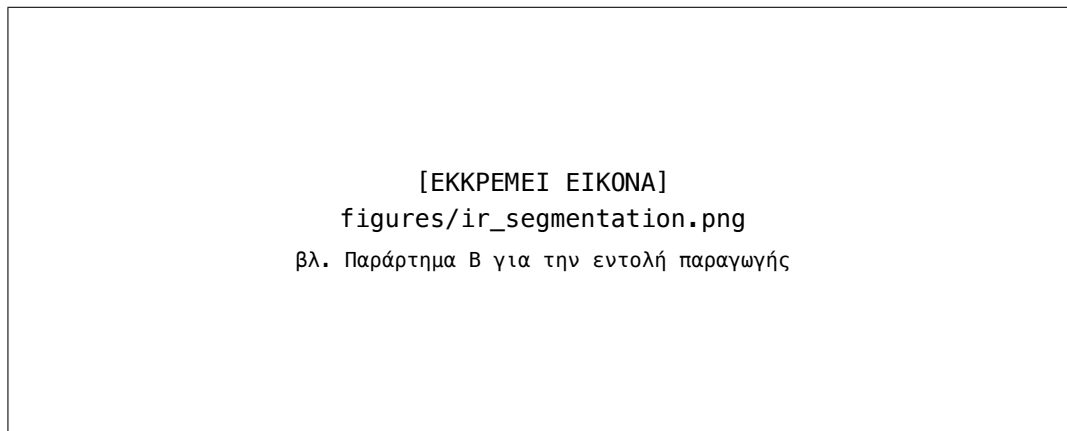


Figure 8: Θερμική κατάτμηση: IR frame (White Hot), μάσκα κατωφλίου 230, τελικό πολύγωνο. Η IR οδός είναι αναίσθητη σε συνθήκες φωτισμού και καπνό, αλλά απαιτεί βαθμονόμηση κατωφλίου ανά αισθητήρα.

5.4 Στάδιο 3: Μορφολογία, πολύγωνο και οπτική τηλεμετρία

Η δυαδική μάσκα καθαρίζεται με μορφολογικό κλείσιμο (ελλειπτικό στοιχείο 4×4) και διπλή διαστολή, ώστε να γεφυρωθούν οι ασυνέχειες που προκαλεί το τρεμόπαιγμα της φλόγας. Ακολουθεί `findContours` (εξωτερικά περιγράμματα) και επιλέγεται το μεγαλύτερο κατά εμβαδόν, με ελάχιστο αποδεκτό εμβαδόν 50 px^2 για απόρριψη θορύβου. Το περίγραμμα μετατοπίζεται από τις συντεταγμένες του `crop` στις συντεταγμένες του πλήρους `frame` και γίνεται το *πολύγωνο φλόγας*.

Από το πολύγωνο υπολογίζονται:

- *Κέντρο βάρους* μέσω ροπών εικόνας: $c_x = m_{10}/m_{00}$, $c_y = m_{01}/m_{00}$ — η είσοδος της γεωαναφοράς (§6).
- *Ρυθμός οπτικής μεγέθυνσης* (optical growth rate): κυλιόμενο ιστορικό 15 δειγμάτων (t_i, A_i) εμβαδού πολυγώνου και εκτίμηση $\dot{A} = \Delta A / \Delta t$ σε px/s μεταξύ πρώτου και τελευταίου δείγματος. Θετικός ρυθμός σηματοδοτεί ενεργά αναπτυσσόμενη εστία — πρώιμο, αμιγώς οπτικό προειδοποιητικό σήμα πριν από κάθε φυσική μοντελοποίηση.

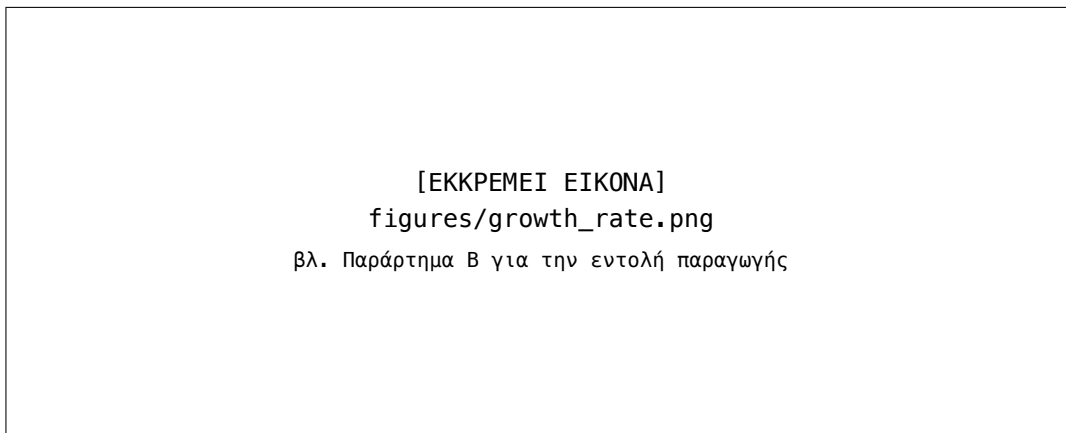


Figure 9: Χρονοσειρά εμβαδού πολυγώνου φλόγας και εξομαλυμένος ρυθμός μεγέθυνσης (px/s) σε δοκιμαστικό βίντεο. Οι κόκκινες περιοχές αντιστοιχούν σε φάσεις ενεργής ανάπτυξης της εστίας.

5.5 Επιδόσεις όρασης

Ο ανιχνευτής τρέχει σε πραγματικό χρόνο σε CPU χάρη στο μέγεθος `nano` του YOLOv8 και στο ότι η κατάτμηση περιορίζεται στα `crops`. Το FPS εμφανίζεται ζωντανά στο `overlay` του `preview` και τυπώνεται ανά `frame` στην κονσόλα.

6 Τρισδιάστατη γεωμετρία I: γεωαναφορά `pixel` $\rightarrow$ `GPS (drone/src/geometry`

Η καρδιά του `drone pipeline` είναι η προβολή ενός `pixel φωτιάς` (u, v) σε απόλυτη γεωγραφική συντεταγμένη WGS-84, με χρήση του μοντέλου `pinhole` κάμερας και της ζωντανής τηλεμετρίας του `drone`. Η υλοποίηση (`FireGeolocator.pixel_to_gps()`) ακολουθεί έξι βήματα με σαφώς ορισμένα συστήματα συντεταγμένων.

6.1 Συστήματα συντεταγμένων και συμβάσεις

- **NED** (North–East–Down): αδρανειακό πλαίσιο κόσμου, πρότυπο MAVLink/autopilot· ο άξονας Z δείχνει προς τα κάτω.
- **Body** (Forward–Right–Down): σωματόδετο πλαίσιο του `drone`.

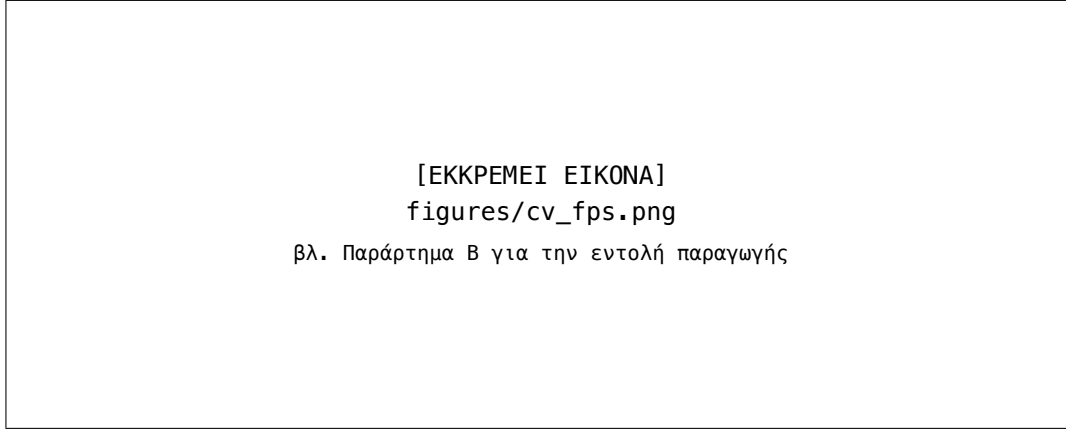


Figure 10: Ρυθμός επεξεργασίας (FPS) του pipeline όρασης ανά διαμόρφωση: YOLOv8n σε imgsz 320 έναντι 640, RGB (πλήρης Bravonoid) έναντι IR (κατωφλίωση), σε CPU laptop-class. Η IR οδός είναι ελαφρύτερη καθώς παρακάμπτει τους διπλούς χρωματικούς χώρους.

- **Cam** (πλαίσιο OpenCV): X δεξιά στην εικόνα, Y κάτω, Z κατά μήκος του οπτικού άξονα προς τη σκηνή.

Η προεπιλεγμένη τοποθέτηση κάμερας είναι *nadir* (κατακόρυφα προς τα κάτω), με τη σταθερή αντιστοίχιση

$$X_{cam} \rightarrow Y_{body}, \quad Y_{cam} \rightarrow X_{body}, \quad Z_{cam} \rightarrow Z_{body} \implies R_{cam \rightarrow body}^{nadir} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Για gimbal με κλίση εμπρός, μια πρόσθετη στροφή $R_y(-\theta_{off})$ προ-συντίθεται μία φορά κατά την αρχικοποίηση (cam_pitch_offset_deg).

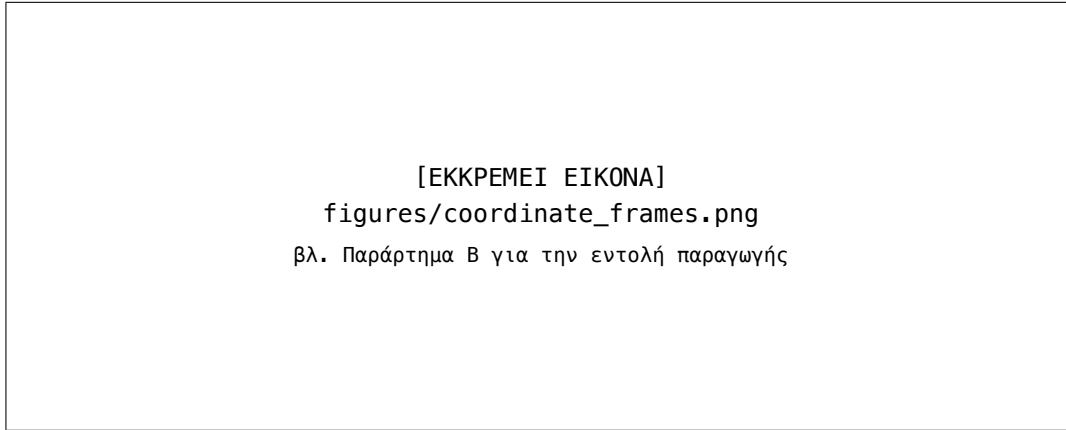


Figure 11: Τα τρία συστήματα συντεταγμένων (NED, Body, Cam) και οι μεταξύ τους στροφές. Σχήμα-ορισμός για όλες τις εξισώσεις της ενότητας.

6.2 Βήμα 1: Αντιστροφή του μοντέλου pinhole

Με εσωτερικές παραμέτρους (intrinsics) f_x, f_y (εστιακή απόσταση σε pixels) και c_x, c_y (κύριο σημείο), το pixel (u, v) απεικονίζεται σε κανονικοποιημένη διεύθυνση στο πλαίσιο κάμερας:

$$x_n = \frac{u - c_x}{f_x}, \quad y_n = \frac{v - c_y}{f_y}, \quad \mathbf{d}_{cam} = (x_n, y_n, 1)^\top.$$

Το διάνυσμα δεν χρειάζεται κανονικοποίηση μέτρου, αφού στη συνέχεια κλιμακώνεται ελεύθερα από την τομή με το έδαφος.

Για την κάμερα του DJI Spark (live feed 1280×720 , διαγώνιο FOV $81,9^\circ$, αισθητήρας $1/2.3''$) η εστιακή απόσταση εκτιμάται αναλυτικά:

$$f = \frac{d_{px}/2}{\tan(\text{FOV}/2)} = \frac{\sqrt{1280^2 + 720^2}/2}{\tan(40,95^\circ)} \approx 846 \text{ px},$$

τιμή που χρησιμοποιείται στο προφίλ spark του main.py ($f_x=f_y=846$, $c_x=640$, $c_y=360$). Το γενικό προφίλ υποθέτει 1920×1080 με $f \approx 980$. Για εργασία ακριβείας προβλέπεται πλήρης βαθμονόμηση με `cv2.calibrateCamera()` (και συντελεστές παραμόρφωσης φακού, που προς το παρόν αγνοούνται — βλ. περιορισμούς, §12).

6.3 Βήματα 2–3: Αλυσίδα στροφών **cam** → **body** → **NED**

Το διάνυσμα περνά πρώτα στο σωματόδετο πλαίσιο, $\mathbf{d}_{body} = R_{cam \rightarrow body} \mathbf{d}_{cam}$, και έπειτα στο NED μέσω γωνιών Euler ZYX (yaw $\psi \rightarrow$ pitch $\theta \rightarrow$ roll ϕ):

$$R_{body \rightarrow NED} = R_z(\psi) R_y(-\theta) R_x(-\phi), \quad \mathbf{d}_{NED} = R_{body \rightarrow NED} \mathbf{d}_{body}.$$

Τα αρνητικά πρόσημα στα θ, ϕ κωδικοποιούν τη σύμβαση προσήμων MAVLink (pitch-up θετικό, roll-right θετικό) σε σχέση με τη φορά περιστροφής των στοιχειωδών πινάκων. Οι R_x, R_y, R_z είναι οι στοιχειώδεις πίνακες στροφής 3×3 (`_Rx`, `_Ry`, `_Rz`).

6.4 Βήμα 4: Τομή ακτίνας με το έδαφος (flat-earth)

Στο NED το Z είναι θετικό προς τα κάτω, οπότε το επίπεδο του εδάφους βρίσκεται σε $z = h$, όπου h το ύψος πτήσης πάνω από το έδαφος (AGL). Η ακτίνα από το drone $\mathbf{P}(t) = t \mathbf{d}_{NED}$ τέμνει το έδαφος όταν $t d_z = h$:

$$t = \frac{h}{d_z}, \quad \Delta N = t d_N, \quad \Delta E = t d_E, \quad \rho = \sqrt{\Delta N^2 + \Delta E^2},$$

με ρ την οριζόντια απόσταση drone–φωτιά. Αν $d_z \leq 10^{-6}$ (ακτίνα οριζόντια ή προς τα πάνω, π.χ. σε ακραίο pitch), δεν υπάρχει έγκυρη τομή και η συνάρτηση επιστρέφει None. Η υπόθεση επίπεδου εδάφους είναι συνειδητή απλοποίηση· η ακριβής λύση θα έτεμνε την ακτίνα με το DEM (βλ. §12).

6.5 Βήμα 5: Μετρικές μετατοπίσεις → **WGS-84**

Οι μετατοπίσεις σε μέτρα μετατρέπονται σε γεωδαιτικές με equirectangular προσέγγιση μικρών γωνιών (μέση ακτίνα Γης $R_E = 6371 \text{ km}$):

$$\Delta \varphi = \frac{\Delta N}{R_E} \cdot \frac{180}{\pi}, \quad \Delta \lambda = \frac{\Delta E}{R_E \cos \varphi_0} \cdot \frac{180}{\pi},$$

όπου φ_0 το γεωγραφικό πλάτος του drone. Για εμβέλεις εκατοντάδων μέτρων το σφάλμα της προσέγγισης είναι αμελητέο ($\ll 1 \text{ m}$) σε σχέση με τις αβεβαιότητες τηλεμετρίας.

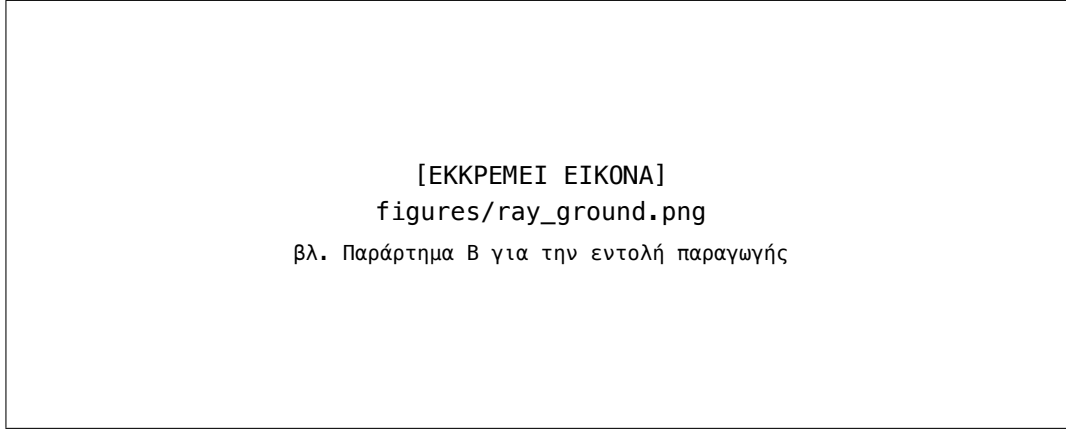


Figure 12: Γεωμετρία τομής ακτίνας–εδάφους: το pixel (u, v) ορίζει ακτίνα στο NED που τέμνει το επίπεδο $z = h$. σημειώνονται η οριζόντια εμβέλεια ρ και η γωνία πλαγιότητας της ακτίνας.

6.6 Βήμα 6: Ευρετική εμπιστοσύνη και ανάλυση σφάλματος

Κάθε αποτέλεσμα συνοδεύεται από δείκτη εμπιστοσύνης

$$c = \text{clip}\left(1 - \frac{\max(|\theta|, |\phi|)}{45^\circ} - \frac{\rho/h}{10}, 0.05, 1\right),$$

που τιμωρεί (α) μεγάλες γωνίες στάσης (όπου τα σφάλματα τηλεμετρίας μεγεθύνονται) και (β) πολύ πλάγιες ακτίνες (μεγάλο ρ/h).

Η θεωρητική ευαισθησία δικαιολογεί τη μορφή του δείκτη: για γωνία πλαγιότητας α από το ναδίρ, η οριζόντια εμβέλεια είναι $\rho = h \tan \alpha$, οπότε ένα σφάλμα στάσης $\delta\alpha$ (rad) προκαλεί σφάλμα θέσης

$$\delta\rho = \frac{h}{\cos^2 \alpha} \delta\alpha,$$

δηλαδή στο ναδίρ ($\alpha=0$) 1° σφάλματος gimbal σε ύψος 100 m δίνει ≈ 1.7 m, ενώ σε $\alpha=60^\circ$ το ίδιο σφάλμα δίνει ≈ 7 m – τετραπλασιασμός. Αντίστοιχα, το χωρικό μέγεθος ενός pixel στο έδαφος (GSD) στο ναδίρ είναι h/f (≈ 0.12 m/px για το Spark στα 100 m), και το βαρομετρικό σφάλμα ύψους περνά αναλογικά στην εμβέλεια: $\delta\rho/\rho = \delta h/h$.

6.7 Τηλεμετρία DJI: από flight record σε γωνίες κάμερας (drone/src/dji_log.py)

Επειδή το DJI Spark δεν εκθέτει MAVLink, το pipeline υποστηρίζει offline λειτουργία: το cached βίντεο του DJI GO 4 συγχρονίζεται με το flight record (.txt $\rightarrow$ CSV μέσω dji-log-parser). Ο DJILogBook χειρίζεται τις ιδιαιτερότητες των exporters:

- *Ασαφής αντιστοίχιση στηλών*: τα ονόματα κεφαλίδων διαφέρουν ανά exporter, οπότε κανονικοποιούνται (πεζά, μόνο αριθμητικά) και ταιριάζονται σε λίστες υποψηφίων ανά λογικό πεδίο.
- *Gimbal αντί σκάφους*: το gimbal σταθεροποιεί την κάμερα ανεξάρτητα από το σώμα, οπότε στη γεωμετρία τροφοδοτείται η στάση του gimbal. Η σύμβαση DJI (0° οριζόντια, -90° κατακόρυφα κάτω) απεικονίζεται στη σύμβαση nadir της γεωμετρίας ως $\theta_{telem} = -(90^\circ + \theta_{gimbal})$, ώστε gimbal $-90^\circ \rightarrow$ ναδίρ και gimbal $0^\circ \rightarrow$ οριζόντιας. Το Spark δεν έχει άξονα yaw στο gimbal, άρα heading κάμερας = yaw σκάφους.
- *Ξετύλιγμα yaw*: το yaw ξετυλίγεται (unwrap) πριν τη γραμμική παρεμβολή, ώστε η παρεμβολή να μην σαρώνει λανθασμένα το άλμα $360^\circ \rightarrow 0^\circ$.

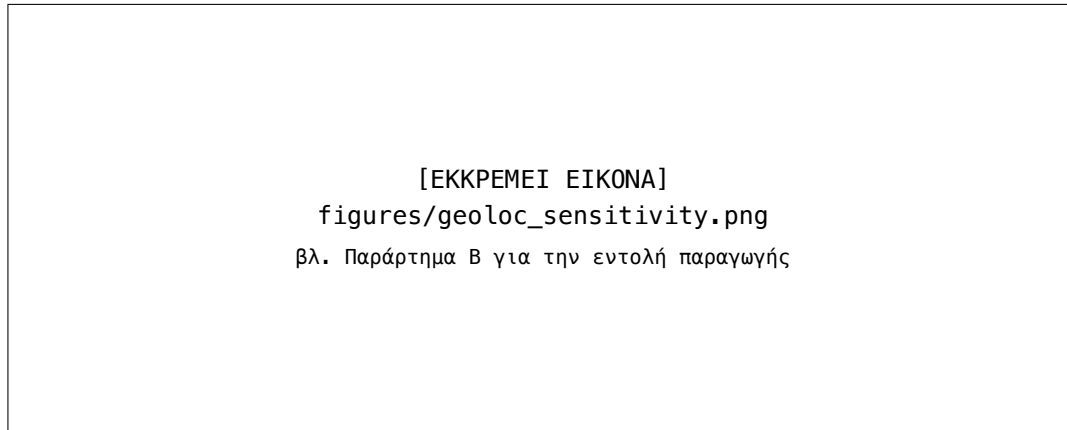


Figure 13: Ευαισθησία γεωαναφοράς: αναμενόμενο σφάλμα θέσης στο έδαφος συναρτήσει γωνίας πλαγιότητας της ακτίνας, για σφάλματα στάσης $0,5^\circ/1^\circ/2^\circ$ και ύψη 50/100/150 m (αναλυτικές καμπύλες $h \delta\alpha / \cos^2 \alpha$). Επάνω δεξιά: ο ευρετικός δείκτης εμπιστοσύνης της υλοποίησης στον ίδιο χώρο.

- *Συγχρονισμός βίντεο–log*: το frame k αγκυρώνεται στη στιγμή που άναψε η σημαία isVideo (έναρξη εγγραφής), με χειροκίνητη διόρθωση `--video-offset` σε δευτερόλεπτα· κάθε frame k αντιστοιχίζεται σε χρόνο $\log t_0 + k/\text{fps}$ και η τηλεμετρία παρεμβάλλεται γραμμικά.
- *Έλεγχοι υγιεινής*: αυτόματη ανίχνευση GPS σε ακτίνια (η Ελλάδα θα διαβαζόταν ως $\varphi \approx 0,66$), απόρριψη γραμμών πριν το GPS lock (0,0), ταξινόμηση κατά χρόνο. Το βαρομετρικό ύψος πάνω από το σημείο απογείωσης χρησιμοποιείται ως AGL — έγκυρο μόνο σε σχετικά επίπεδο έδαφος γύρω από το σημείο εκτόξευσης.

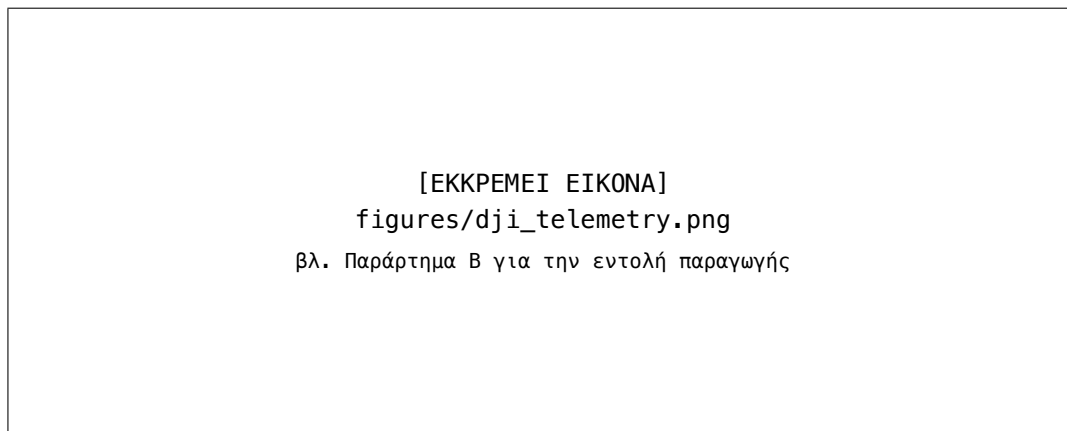


Figure 14: Τηλεμετρία flight record (πτήση 2026-07-08): ύψος AGL, gimbal pitch και yaw συναρτήσει χρόνου, με σκιασμένα τα διαστήματα ενεργής εγγραφής βίντεο (isVideo) στα οποία αγκυρώνεται ο συγχρονισμός frame–τηλεμετρίας.

6.8 Ενορχήστρωση και καταγραφή (drone/src/main.py)

Ο «ελεγκτής κυκλοφορίας» ενώνει όραση και γεωμετρία πίσω από μια αφηρημένη διεπαφή DroneInterface με τρεις υλοποιήσεις: (1) MAVLinkInterface για ArduPilot/PX4 (τηλεμετρία από GLOBAL_POSITION_INT+ATTITUDE στα 10 Hz, μη-μπλοκάρουσα αποστράγγιση μηνυμάτων, βίντεο από USB/RTSP), (2) DJILogInterface (offline replay όπως παραπάνω), (3) VideoFileInterface με mock τηλεμετρία για ανάπτυξη. Κάθε

ανίχνευση καταγράφεται στο `data/fire_gps_log.csv` (χρονοσφραγίδα UTC, θέση/στάση drone, εκτιμώμενη θέση φωτιάς, εμβέλεια, εμπιστοσύνη, pixel), ώστε οι εντοπισμοί να μπορούν μελλοντικά να τροφοδοτούν αυτόματες εναύσεις στον προσομοιωτή.

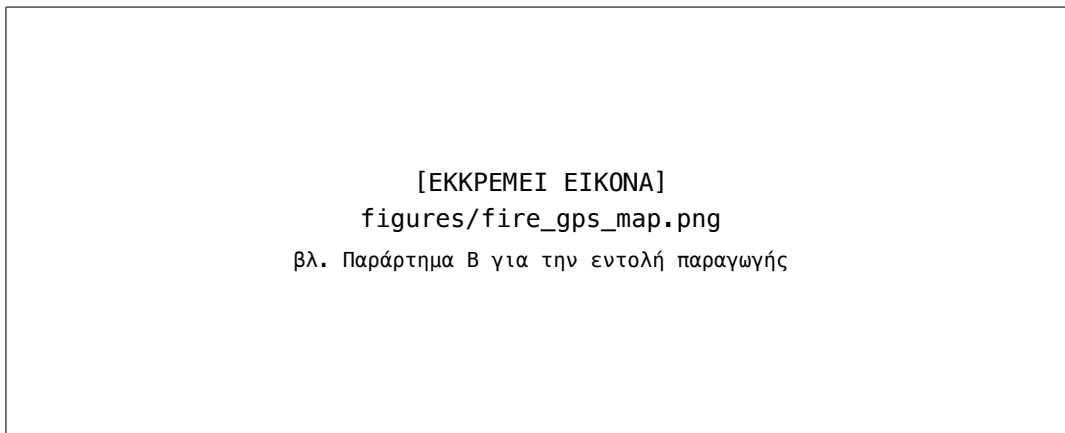


Figure 15: Γεωαναφερμένες ανιχνεύσεις από το `fire_gps_log.csv` πάνω σε δορυφορικό υπόβαθρο: ίχνος πτήσης του drone (μπλε), εκτιμημένες θέσεις φωτιάς χρωματισμένες κατά δείκτη εμπιστοσύνης, και η διασπορά της συστάδας εκτιμήσεων γύρω από την πραγματική εστία.

7 Τρισδιάστατη γεωμετρία II: ανακατασκευή και απεικόνιση αναγλύφου

7.1 Flask API πλεγμάτων `mesh_api.py`

Το endpoint `POST /mesh` δέχεται `{lat, lon, bbox, wind}` και επιστρέφει τρία διαδραστικά Plotly figures (JSON) συν πλέγμα ανέμου:

- *Γεωμετρικά σωστό αποτύπωμα*: η περιοχή είναι φυσικά τετράγωνη 2×2 km — το γεωγραφικό μήκος διορθώνεται κατά $1/\cos \varphi$ ώστε πλάτος και ύψος σε μέτρα να ταυτίζονται (`_square_bounds()`), αντί για «τετράγωνο» σε μοίρες που στην Ελλάδα θα ήταν $\sim 21\%$ πλατύτερο απ' ό,τι ψηλό.
- *Πηγές δεδομένων μέσω `slippy tiles`*: μετατροπή $(lat, lon) \rightarrow (x, y, z)$ με τη Web-Mercator σχέση $y = (1 - \operatorname{asinh}(\tan \varphi)/\pi) 2^{z-1}$. DEM από Terrarium tiles σε zoom 14 ($\approx 9,5$ m/px) και δορυφορική υφή ESRI σε zoom 17 ($\approx 1,2$ m/px).
- *Τρεις αναπαραστάσεις*: (α) DEM surface με ντυμένη δορυφορική υφή, (β) νέφος σημείων (έως 4.000 σημεία), (γ) TIN (τριγωνοποίηση Delaunay) — και οι τρεις σε μετρικούς άξονες X/Y/Z με aspect ratio $z = 0,35$ για ρεαλιστική υπερύψωση.
- *Βέλη ανέμου*: πλέγμα 10×10 διανυσμάτων από το `air/air.py` (πλήρης αλυσίδα διόρθωσης της §2.3) όταν δίνεται ταχύτητα/διεύθυνση, αλλιώς ομοιόμορφο fallback.
- Συνθετικό fallback DEM όταν αποτύχει η λήψη πραγματικού.

7.2 Live 3D προβολή προσομοίωσης `simulation_3d_view.py`, `viz/`

Το `simulation_3d_view.py` εξάγει την τρέχουσα περιοχή σε NPZ και εκκινεί subprocess viewer με live ενημέρωση κατάστασης μέσω αρχείου: το μέτωπο της φωτιάς αποδίδεται ως overlay πάνω στο τρισδιάστατο ανάγλυφο όσο τρέχει η προσομοίωση. Ο φάκελος `viz/` περιέχει τους standalone visualizers (2D και 3D).

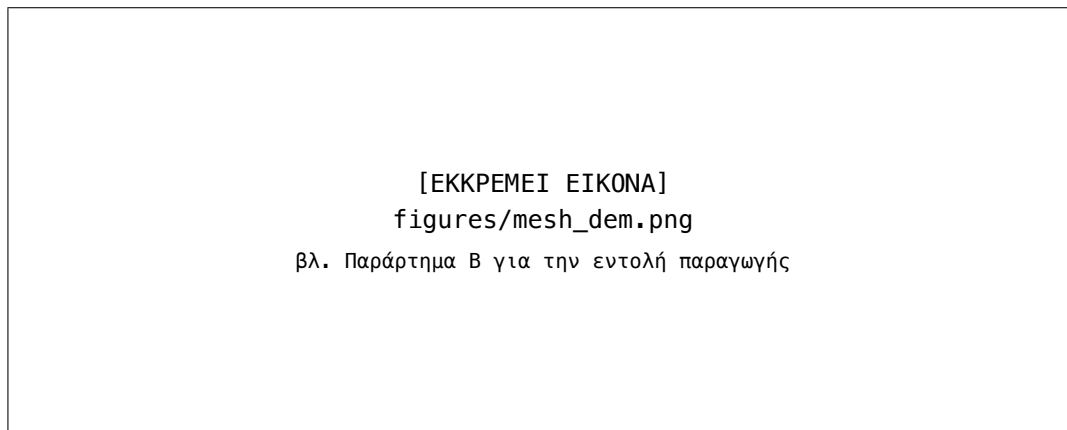


Figure 16: Plotly DEM surface 2×2 km με δορυφορική υφή ESRI z17 και βέλη διορθωμένου ανέμου (10×10). Screenshot από το 3D popup του UI.

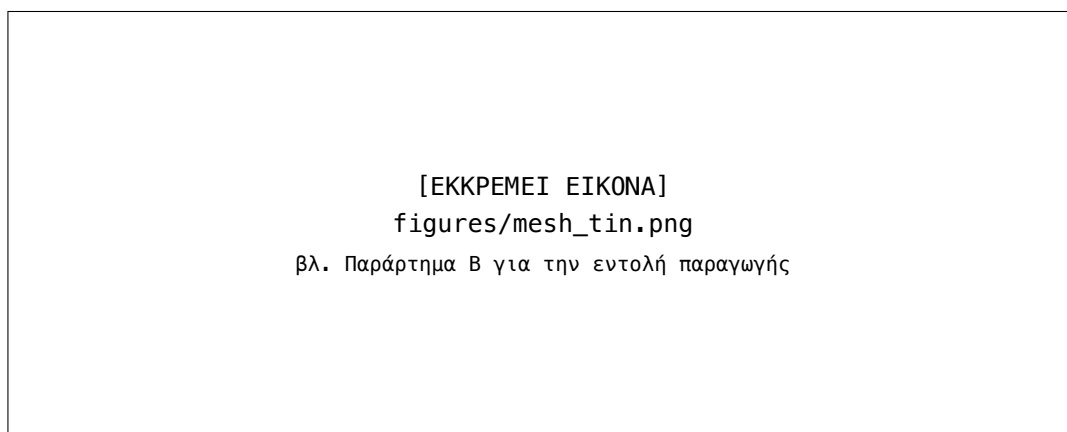


Figure 17: Το ίδιο ανάγλυφο ως TIN (τριγωνοποίηση Delaunay) και ως νέφος σημείων· οι τρεις αναπαραστάσεις επιστρέφονται μαζί από το POST /mesh.

8 Server προσομοίωσης `server.py`

WebSocket server (port 8765) που ενοποιεί όλα τα υποσυστήματα:

- *Πρωτόκολλο*: `init` (περιοχή, εναύσεις, καιρός, προαιρετικό optimizer mode) → `init_ack` + ροή frame (RLE GeoJSON καιγόμενων/καμένων· 7,3 ms + 104 KB ανά frame σε 800²)· ξεχωριστά sessions `init_optimizer` και `init_hindcast`.
- *Επεμβάσεις*: `containment line` (βαθμωτή απόσβεση + νερό + υγρασία, με φθορά στον χρόνο), `firebreak` (σκληρό φράγμα που μπλοκάρει και τις καύτρες στη διαδρομή), `water drop/brush` (Monte-Carlo σταγόνες).
- *Ωριαίος καιρός*: ενημέρωση από ERA5 όταν δίνεται ημερομηνία, με επανεφαρμογή της βαθμονόμησης στο `p_spread` μετά από κάθε `rebuild`.
- *Optimizer continue-modes*: `ground_truth` (η περίμετρος της μάσκας καίγεται, το εσωτερικό καμένο) και `best_params_sim` (επαναφορά του παγωμένου best fire)· και τα δύο ξεκινούν *παγωμένα* στην ηλικία της μάσκας.
- Ιστορικό snapshots για scrubbing, cell-explanation (Alt+click, πλήρης ανάλυση Rothermel ανά κελί), rect-analysis, ενημέρωση microclimate στο τέλος.

9 Web διεπαφή `sandbox.html`

Ενιαία σελίδα (~5.500 γραμμές) με χάρτη MapLibre και αριστερό rail με συρτάρια: Setup & Weather, Tools (εναύσεις, μάσκα αληθείας, νερό, containment, firebreak ζώνες με `save/load — firebreak.py`), Optimizer (PSO + συνέχιση προσομοίωσης), Hindcast Validation (3 προκαθορισμένες πυρκαγιές), Microclimate Memory, Fire Events, Threat Advisor. Η απόδοση της φωτιάς συνδυάζει GeoJSON layers με canvas glow/καπνό (particles), wind streamlines από το πεδίο του server, ηχητικό feedback, θέμα light/dark (Catppuccin) και 41 προγραμματιστικά παραγόμενα PNG εικονίδια (`assets/gen_icons.py`). Τα frames συγχωνεύονται σε `requestAnimationFrame`· υπάρχει scrubber ιστορικού και 3D popup (§7).

10 Σύμβουλος απειλών (Threat Advisor) `pipeline/threat_advisor.py`

Live εκτίμηση κινδύνου ανά οικισμό:

- Απόσταση από το ενεργό μέτωπο μέσω distance transform της μάσκας καύσης, EMA ταχύτητας προσέγγισης, ETA = απόσταση/ταχύτητα· κλιμάκωση `safe` → `watch` (≤ 6 h) → `warning` (≤ 2 h ή 1,5 km) → `critical` (≤ 30 min ή 500 m) → `impact`.
- Γλωσσικές αναφορές *μόνο σε κλιμάκωση* + προτάσεις: σημείο ρίψης νερού στο 35% της διαδρομής μετώπου→οικισμού, γραμμή ανάσχεσης κάθετη στον άξονα προσέγγισης.
- Διαδρομές διαφυγής: Dijkstra σε γράφο OSM (~100k κόμβοι στην Εύβοια) με ποινές εγγύτητας φωτιάς (≤ 300 m = αποκλεισμένο)· έξοδοι = κόμβοι στο εξωτερικό 4% του χάρτη· ειλικρινές fall-back «shelter in place» όταν όλοι οι δρόμοι κόβονται.
- Εκτέλεση κάθε ≥ 4 s wall σε worker thread πάνω σε snapshot του state (30–50 ms). Παγίδα: ελληνικά χωριά μοιράζονται ονόματα — όλα τα κλειδιά είναι `name@lat, lon`.

Συμπληρωματικά, το `fire_info_panel.py` μετατρέπει την τηλεμετρία της προσομοίωσης σε φυσική γλώσσα (ορόσημα έκτασης, κατεύθυνση/ταχύτητα εξάπλωσης με δειγματοληψία εδάφους ~150 m μπροστά από το μέτωπο, αναφορές καιρού και επεμβάσεων).

11 Εκκίνηση και βοηθητικά

`main.py`: single-command εκκίνηση των τριών servers (HTTP 8000 / WebSocket 8765 / Mesh 5000) και άνοιγμα browser. `gui_launcher.py`: Tkinter GUI του hindcast pipeline βήμα-βήμα. `firebreak.py`: αποθήκευση/φόρτωση αντιτυρικών ζωνών σε CSV/JSON. `config.py`: καθολικές σταθερές (πλέγμα, BURN_TIME_STEPS, καιρικές προεπιλογές).

12 Επικύρωση, περιορισμοί και μελλοντικές επεκτάσεις

12.1 Τι έχει μετρηθεί

Το πλήρες παράρτημα μετρήσεων βρίσκεται στο `report_data.tex/.pdf`: hindcast sweep 13 τρεξιμάτων, καμπύλες φυσικής, αποτελεσματικότητα επεμβάσεων (αντιτυρική ζώνη 53–54%, ρίψεις 46%, γραμμές ανάσχεσης 6–11% με πλήρη απόδοση της αποτυχίας στις καύτρες), σύγκλιση PSO, στατιστικά έργου.

12.2 Γνωστοί περιορισμοί

- *Προσομοίωση*: απουσία μοντέλου ενεργητικής κατάσβεσης (συστηματική υπερπρόβλεψη), επιφανειακή μόνο φωτιά (το crown fire απορροφάται από πολλαπλασιαστές), ο ERA5 0,25° ομαλοποιεί τον άνεμο, το cumulative scoring διογκώνει το IoU των όψιμων ημερών, υπερβολή spotting σε κελιά 25 m.
- *Όραση*: τα βάρη YOLO δεν έχουν fine-tuned σε ελληνικό υλικό· οι χρωματικοί κανόνες έχουν σταθερά κατώφλια (ευαίσθητα σε white balance)· το IR κατώφλι 230 απαιτεί ρύθμιση ανά κάμερα· ο ρυθμός μεγέθυνσης είναι σε px/s, όχι m²/s (θα απαιτούσε προβολή του πολυγώνου στο έδαφος).
- *Γεωμετρία*: υπόθεση επίπεδου εδάφους (σε πλαγιές η τομή με DEM θα διόρθωνε συστηματικό σφάλμα εμβέλειας), αγνόηση παραμόρφωσης φακού, βαρομετρικό AGL έγκυρο μόνο κοντά στο σημείο απογείωσης, equirectangular προσέγγιση (αμελητέα στα μεγέθη μας).

12.3 Πιθανές επεκτάσεις

Ωριαίος καιρός εντός του hindcast παραθύρου· proxy κατάσβεσης· scoring χωρίς το assimilated seed· πληθυσμός οικισμών στην ιεράρχηση του Threat Advisor· *σύνδεση των drone ανιχνεύσεων με αυτόματη έναυση στον προσομοιωτή* (το `fire_gps_log.csv` έχει ήδη το απαιτούμενο σχήμα)· τομή ακτίνας με DEM αντί για flat-earth· βαθμονόμηση κάμερας με chessboard· μετατροπή του optical growth σε φυσικές μονάδες μέσω της γεωαναφοράς όλου του πολυγώνου (όχι μόνο του κέντρου βάρους).

A Παράρτημα A: Πίνακες μετρήσεων

Βλ. συνοδευτικό `report_data.tex` (μεταγλωττίζεται αυτόνομα ή τα tables του μπορούν να γίνουν `\input` εδώ).

B Παράρτημα B: Οδηγίες αναπαραγωγής σχημάτων

Όλα τα σχήματα αναμένονται στον φάκελο `report/figures/` με τα ονόματα του πίνακα. Μέχρι να τοποθετηθούν, το έγγραφο τυπώνει πλαίσια placeholder.

Αρχείο	Τρόπος παραγωγής
ros_curves.png	Σουίτα χαρακτηρισμού φυσικής (sweep ROS-άνεμος/κλίση/υγρασία)· τα δεδομένα υπάρχουν στο report_data.tex, ενότητα C.
wind_field.png	Screenshot των wind streamlines στο UI, ή απευθείας quiver plot από το air/air.py πριν/μετά τη διόρθωση.
pso_convergence.png	Καταγραφή best-IoU ανά γενιά από το particle_optimizer.py (5 seeds, με/χωρίς microclimate warm-start).
hindcast_evia.png	python gui_launcher.py ή απευθείας hindcast_optimizer.py — το τετράπτυχο PNG της plot_results().
cv_pipeline.png	Σχεδιάγραμμα (draw.io/TikZ) βάσει §5.
yolo_detections.png	Μωσαϊκό από υπάρχοντα frames στο drone/data/vision_check/ (frame_*.png, node-detect_*.png).
bravonoid_masks.png	Μικρό script που καλεί full_bravonoid_segmentation() σε ένα crop και σώζει τα ενδιάμεσα στάδια δίπλα-δίπλα.
ir_segmentation.png	python drone/fire_tracker.py σε test_ir.mp4 με αποθήκευση σταδίων της thermal_segmentation().
growth_rate.png	Plot της χρονοσειράς area_history από τρέξιμο του fire_tracker.py.
cv_fps.png	Bar chart από τις τιμές FPS του overlay σε 4 διαμορφώσεις (320/640 × RGB/IR).
coordinate_frames.png, ray_ground.png	Σχεδιαγράμματα (TikZ/draw.io) βάσει §6.
geoloc_sensitivity.png	Αναλυτικές καμπύλες $h \delta\alpha / \cos^2 \alpha - 20$ γραμμές matplotlib, δεν απαιτεί πτήση.
fire_gps_map.png	Plot του drone/data/fire_gps_log.csv πάνω σε δορυφορικό υπόβαθρο (folium/contextily).
dji_telemetry.png	Plot στηλών height/gimbal pitch/yaw/isVideo από το ExportCSV_2026-07-08_[15-30-54].csv μέσω DJILogBook.
mesh_dem.png, mesh_tin.png	Screenshots από το 3D popup του UI (POST /mesh στο port 5000).

Table 1: Αντιστοίχιση σχημάτων–εντολών παραγωγής.